

GEO1015 - Digital Terrain Modelling

Assignment 3 - The best shape detection algorithm is...

Names:

Carmem E. Felix Aires 4325893

Nicolette Corsel 5309859

Xinya Bi 6195350

Xu Wang 6235379

RANSAC

Describe your implementation. How does it divert from what is explained in the book? Did you modify it so that complex cases are handled? Did you encounter any unexpected results?

The implementation of RANSAC follows the basic structure as presented in the book with a few modifications:

1. Obviously, three points are selected to form a plane (instead of two points for a line) and a plane equation is used, since the goal here is to identify planes.
2. A KDTree for the points was created for efficient computation.
3. Instead of randomly selecting three points at the same time in the entire dataset, only the first point is randomly selected and the following two points must be the neighbours of this first point. Here a new parameter is added: `neighborhood_radius_1stplane`. And the function `query_ball_point` from `scipy` is used.

This was a way to solve one major problem of the RANSAC algorithm:

The problem: The “raw” algorithm starts by selecting three random points in the whole dataset. **In practice and for the BK city**, these three points can be very far apart, therefore the plane formed by them tends to be rather horizontal, but not necessarily corresponding to the horizontal planes of the building, “slicing” the building several times (see figure 1). This is because the parameter `min_score`, which defines the minimum number of points for a plane to be taken into account, **is easily reached for a building as large as BK**. However, for small buildings, this does not seem to happen with the same frequency, since the number of points in a real plane is greater than the points in a “slice” of the building (see figure 2).

Selecting the second and third points that are closer together (e.g. 2m-3m), minimises the chance of false horizontal planes being detected, as only corner points would cause this problem. (see figure 3).

Elapsed time for the computation: 3,41 min

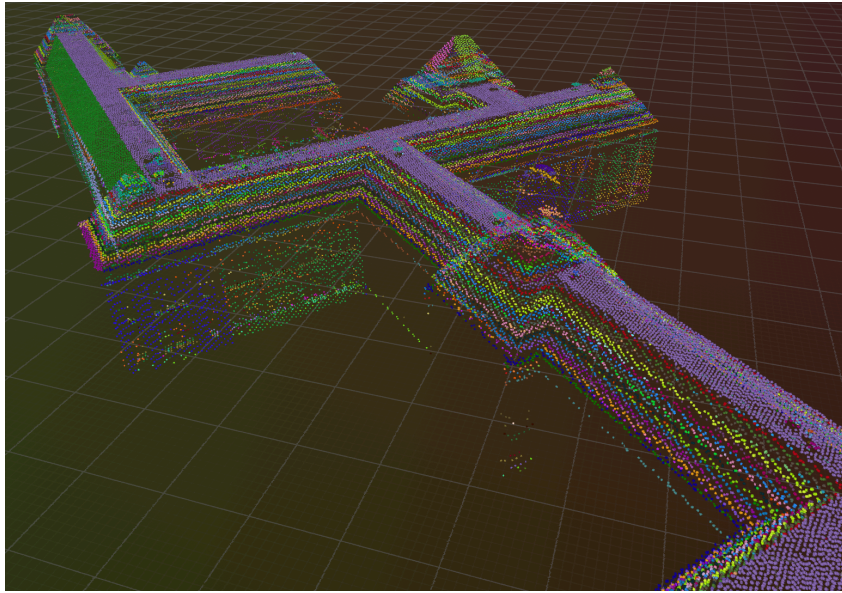


Figure 1: the “raw” algorithm has a major drawback which is the detection of several horizontal lines instead of planes.

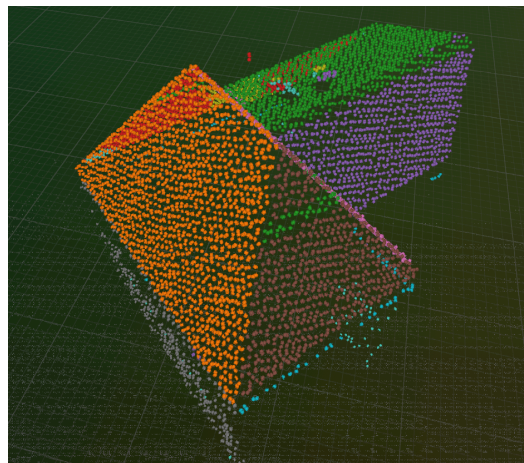
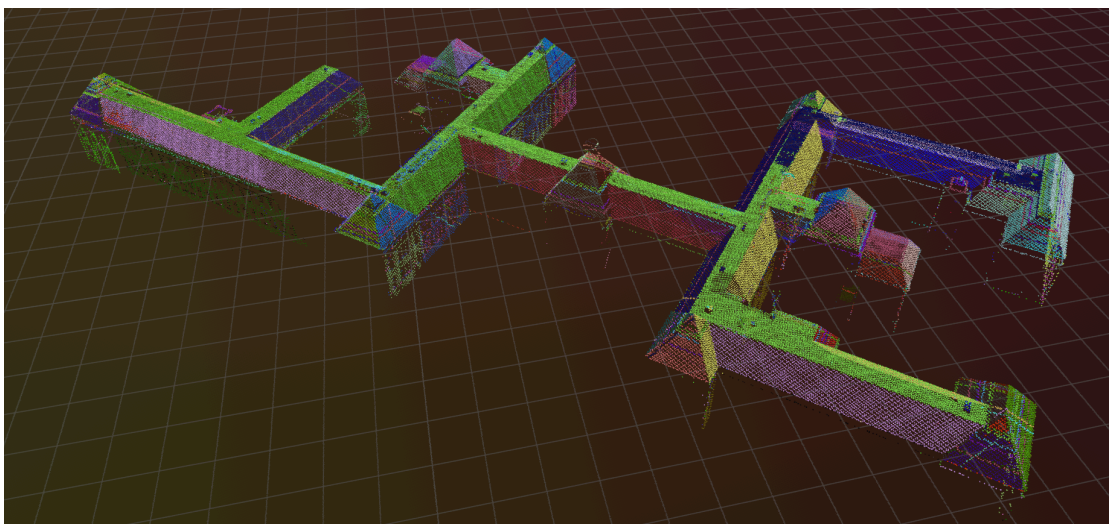
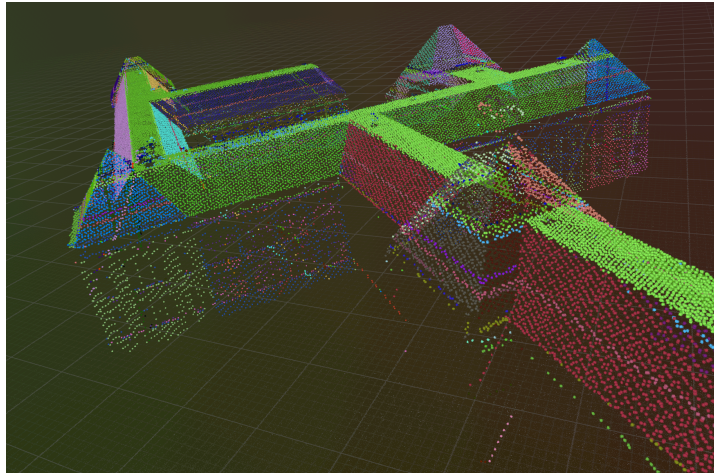


Figure 2: for smaller buildings this is less of a problem, since horizontal slices will have less points than the planes, therefore this is easily adjusted with the `min_score` parameter.



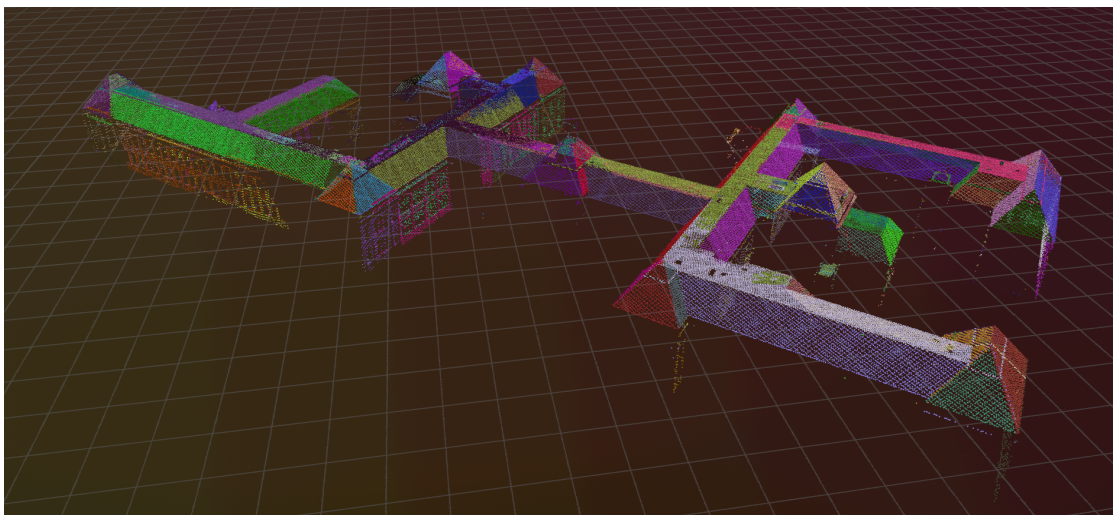


```
{
  "input_file": "bk.laz",
  "RANSAC": {
    "k": 1000,
    "min_score": 200,
    "epsilon": 0.15,
    "neighborhood_radius_1stplane": 2
  }
}
```

Figure 3: BK building after change number 3 and the parameters used.

4. The problem of horizontal slices was minimized after step 3, however it does not avoid it completely, since points that are very far apart were still being added to the same planes. Another limiting distance parameter was therefore necessary. The parameter is the maximum distance a point can be added to a plane, given the plane centroid calculated from the 3 initial points. The parameter is named `neighborhood_radius_allplanes`. For BK-City a good distance is 45 - 47 m, since the largest planes are around 90m (see Figure 4). This parameter would be tricky to set in a dataset with multiple and heterogeneous building sizes.

Elapsed time for the computation: 3,9 min




```
{
  "input_file": "bk.laz",
  "RANSAC": {
    "k": 1000,
    "min_score": 200,
    "epsilon": 0.15,
    "neighborhood_radius_allplanes": 45,
    "neighborhood_radius_1stplane": 2
  }
}
```

Figure 4: BK-Building after amendment number 4 and the used parameters.

A side effect of this adjustment is that the planes that are longer than 90m are subdivided. This was adjusted by a function `merge_similar_planes` which merges plane segments that have similar normal vectors within an angular tolerance and the mean point of one plane lies on the other plane within a given threshold. This function needs further adjustment, since a few planes are not yet added together, probably because of a coding problem. Another necessary improvement is avoiding that disjointed coplanar planes are added together (see figure 5).

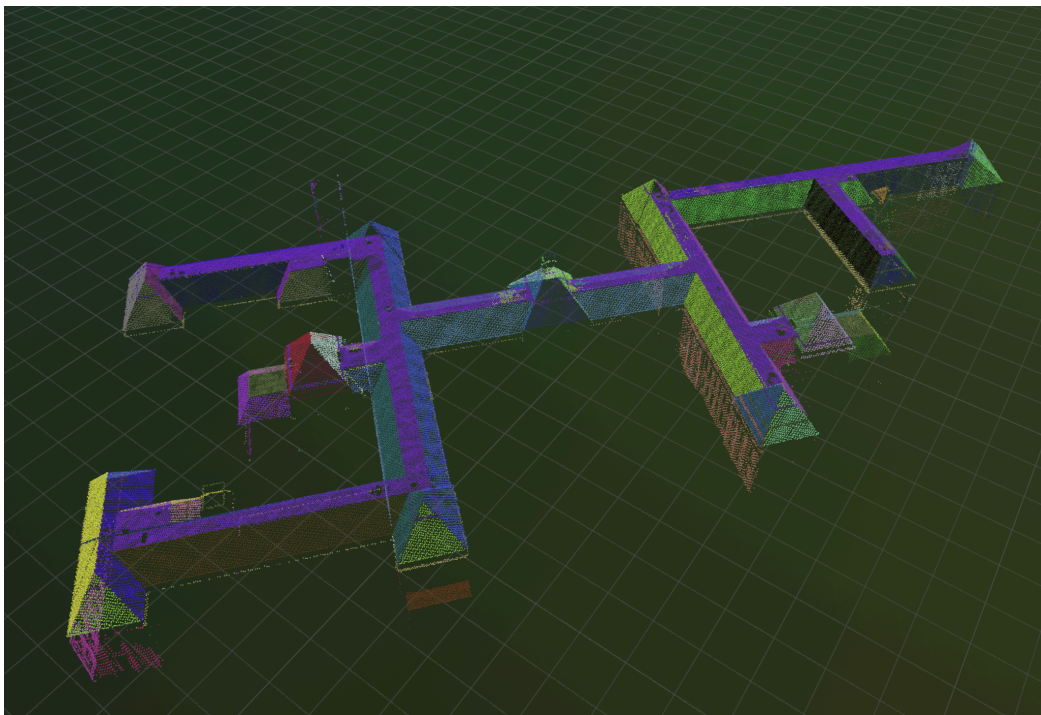


Figure 5: The final segmentation result of BK-city for RANSAC.

5. Neighbouring points that fit into a plane and are visited first are still added to the incorrect plane (see Figure 6). One possible solution would be to simply filter them out, as it was observed that the boundary of the BK city planes was usually already defined, which is sufficient to define a plane. Another way would be to calculate the intersections of the planes and remove the points that fall in there. Due to time constraints this was not possible to be implemented..

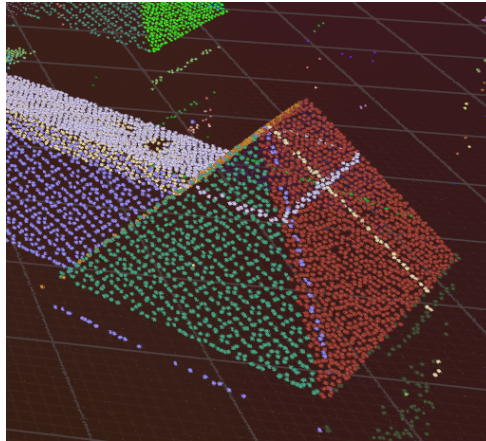


Figure 6: Points in neighbouring planes (closer than 47m) that are first visited are added to planes erroneously.

What parameters are necessary and exposed to the user? Show the results and provide the best parameters for the BK-City dataset. Also, how did you choose the optimal parameters?

For BK-City the best parameters are the following:

```
{
  "input_file": "bk.laz",
  "RANSAC": {
    "k": 1200,
    "min_score": 200,
    "epsilon": 0.15,
    "neighborhood_radius_allplanes": 47,
    "neighborhood_radius_1stplane": 3
  }
}
```

As explained before two new parameters were added.

`neighborhood_radius_1stplane` and `neighborhood_radius_allplanes`.

Please refer to the previous question for a detailed explanation.

`k` (number of iterations): 1200 - 1500 iterations results in very good plane fits. With less than 1000, the resulting planes are much more segmented and less accurate. Above this amount of iterations, the planes are very good, however runtime starts growing a lot, since the time complexity of RANSAC is very dependent on it ($O(kn)$).

`Min_score` (minimal amount of points for a plane to be defined): As some walls contain a small number of points, the `min_score` must be very low: 200. If these small walls can be ignored (because it is a problem of this dataset that the walls have a really low point density or do not contain any points), a `min_score` of 500 would be much better for larger planes, as the planes would be less segmented and the computation time would be reduced. In this case the parameter `neighbourhood_radius_allplanes` parameter would also need to be set to 50m, as larger planes will be missing some points.

Epsilon(distance of a point to a plane, so that the point is added to it): An epsilon of 15 cm gives better results, because some planes in BK-City are very close to each other, so larger distances may cause them to be added to the wrong plane.

List the pros/cons of the method.

Pros	Cons
Implementation is straightforward. Only a few modifications were necessary, although they were very necessary because the raw algorithm result was unusable.	Given the random nature of the algorithm, the points that are first visited might be added to incorrect planes. Fixing this would require a lot of modifications and further visiting of points to be fixed.
Easy parameter setting for the user	Also because of the random nature, the first three points visited define the surface, but this might be slightly off the best surface. This is minimized by the number of iterations
Outliers are well removed	This is a rather slow algorithm

Region growing

Describe your implementation. How does it divert from what is explained in the book? Did you modify it so that complex cases are handled? Did you encounter any unexpected results? Eg. planes fitted to points where you don't want it. Show examples and explain why that happened.

The region growing method follows the structure presented in the book, with several modifications to enhance its robustness and completeness. Here are the key modifications:

1. Calculation of Normals and Geometric Features. I implemented the PCA method for normal calculation as suggested in Appendix C and I computed the geometric feature to include: linearity $(\lambda_1 - \lambda_2) / \lambda_1$, planarity $(\lambda_2 - \lambda_3) / \lambda_1$, sphericity λ_3 / λ_1 . These additional features provide richer geometric characteristics of each point's neighborhood
2. Seed Point Selection Strategy. The book suggests selecting seeds based on plane fitting error, it doesn't provide specific criteria. So I implemented a quantitative selection criteria based on geometric features: prioritized points with high planarity values and lower linearity and sphericity compared to planarity. This ensures seed points have stable normal vectors, which is crucial as they form the foundation for subsequent region growing
3. Core Region Growing Algorithm. It followed the book's algorithmic structure. The only modification I did is create a boolean array to track processed points. This prevents redundant point processing and can improve computational efficiency.
4. Processing of Unclassified Points(**Optional**). This is a new phase not present in the book's methodology. Because after step 3, edge and corner points often remain

unclassified. The implementation includes: computing plane equations for all detected regions, assigning remaining unclassified points to the nearest plane based on point-to-plane distance. This enhancement improves the segmentation completeness, especially on architectural features with edge and corner points.

But it will introduce new problems, spatially disconnected planes will be joined together. So this step is optional, and I don't advise users to take it.

Unexpected Results

1. During implementation, smaller elevated surfaces, particularly ventilation systems on rooftops, often remained unclassified. This limitation can be attributed to several factors:
 - Point density constraints : Smaller surfaces typically have fewer points, the sparse point distribution makes it difficult to establish reliable local neighborhoods, and region growing algorithm relies heavily on local neighborhood connectivity, areas with low point density struggles to find sufficient neighbors for normal estimation, establish continuous regions through the connectivity among points, and meet the minimum region size threshold.
 - Seed point selection issues: The current seed selection strategy, which prioritizes high planarity, may overlook small surfaces. Because there may be few or no points that meet the high planarity threshold, resulting in no seed point being selected on the surface, no regions can be grown from these surfaces.

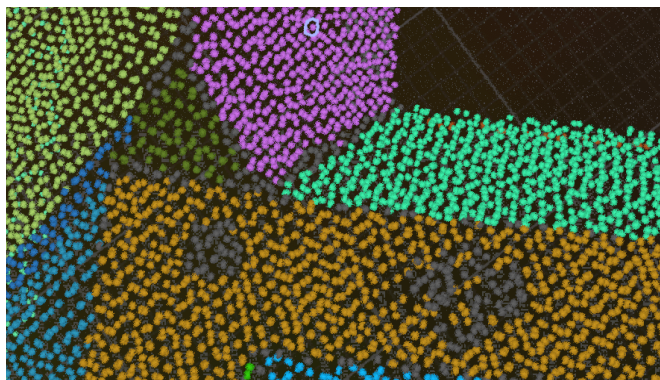


Figure 1: unclassified small surfaces

2. Unclassified edge/ corner points:
 - Main reasons: Normal vectors are unstable at edges and corners because searching for neighbor points in these locations includes points from multiple surfaces, resulting in unreliable normal estimation where they don't align with any actual surface.

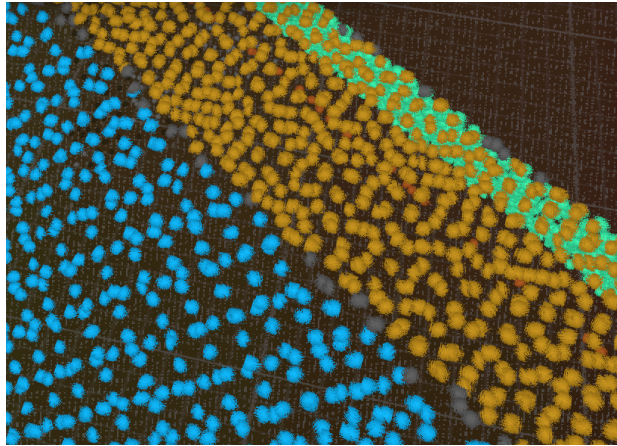


Figure 2: unclassified points on edges/corners

The following problems only occurs when taking step 4

3. Physically separate surfaces incorrectly merged into single region
 - Possible Causes:
 - Bridge Points: Points with similar normals creating invalid connections
 - Seed Point Distribution: Multiple seeds on similar but disconnected surfaces
 - Failed Solutions:
 - Distance thresholds on region size
 - Neighborhood search parameter adjustments
 - Seed point selection constraints
4. Unclassified Points Problem:
 - Possible causes:
 - Points exceeding distance threshold to all planes
 - Data noise/outliers
 - Intersection points between multiple planes that exceed thresholds

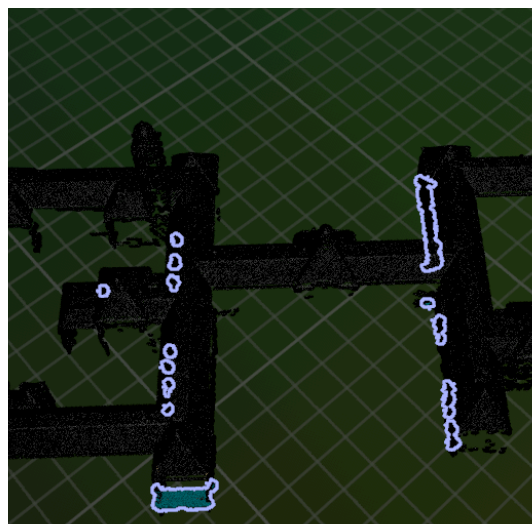


Figure 3: spatially disconnect points being joined to the same plane

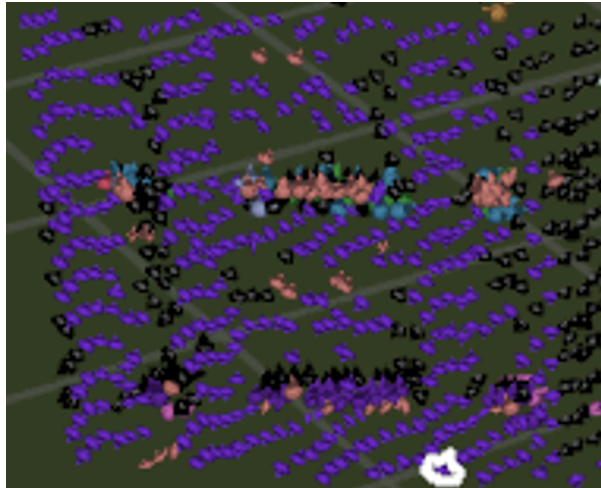


Figure 4: unclassified points problems (the black points are unclassified ones)

What parameters are necessary and exposed to the user? (mention if you added more than the ones given in the starting code)

```
"RegionGrowing": {
  "k": integer,
  "max_angle": float,
  "minimum_region_size": integer,
  "minimum_planarity": float
},
```

1. `k`: Number of neighboring points used for normal vector calculation and geometric feature computation.
2. `max_angle`: Angular threshold for region growing. Controls whether a point belongs to the current region based on normal vector comparison.
3. `minimum_region_size`: Minimum number of points required to form a valid plane region. This parameter is dependent on point cloud density.
4. `minimum_planarity`: Threshold for seed point selection based on geometric features. Higher values indicate stronger planarity characteristics.

Show the results and provide the best parameters for the BK-City dataset. Also, how did you choose the optimal parameters?

```
"RegionGrowing": {
  "k": 15,
  "max_angle": 20.0,
  "minimum_region_size": 50,
  "minimum_planarity": 0.8
}
```

1. `k`: this is chosen based on point cloud density, if the point cloud is unevenly distributed, it would be difficult to choose, but `k = 15` handles the bk building well, it is either not taking too many neighborhood points or too little

2. `max_angle`: the original `max_angle` is a bit too strict, 10 degree will lead to many edge and corner points not classified, but 30 degree is too large, it will result in surface merging problems.
3. `minimum_region_size`: I choose 50 because it can achieve a balance between keeping the details while not over-segment the planes
4. `minimum_planarity`: 0.8 is to strictly filter out points that have good planarity rather than sphericity and linearity.

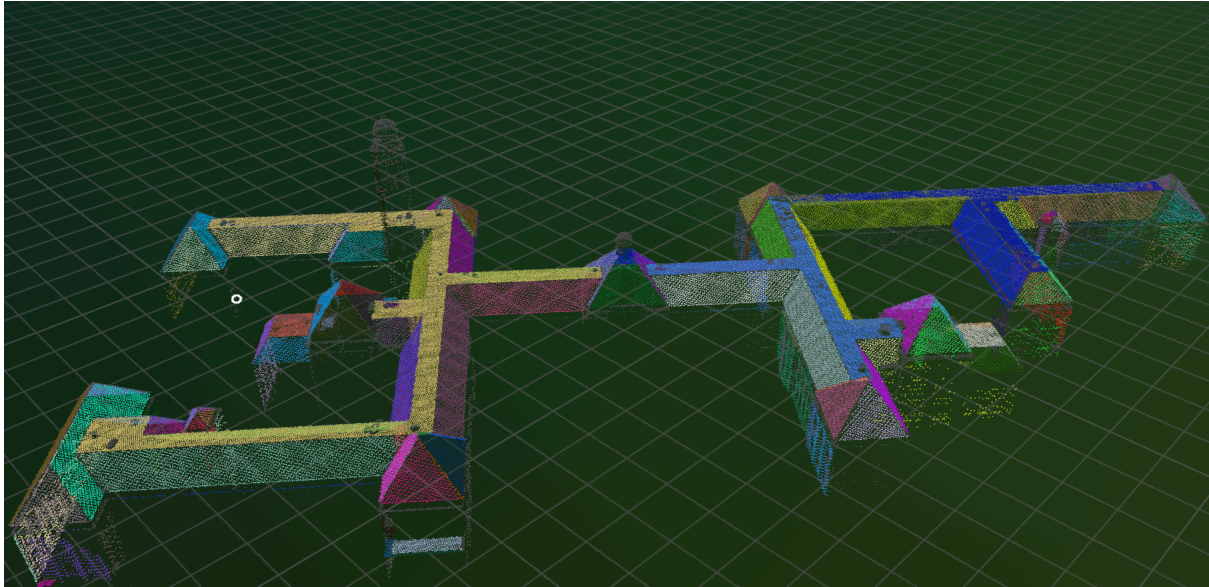


Figure 5: The final segmentation result of BK-city for Region Growing

List the pros/cons of the method.

Pros	Cons
Efficiency: using kd tree to process the data, thus the processing speed is pretty fast	Parameter Sensitivity: Too many parameters for users to set, especially the <code>max_angle</code> , number of neighbors, region size, require good knowledge of the dataset, especially the point density. Because different parameters will lead to different classification results, and for each dataset, it requires different parameters.
Natural Segmentation: Produces regions that follow natural boundaries of surfaces	Robustness Problems: Region growing isn't very stable in regions that have uneven point density, edges, intersections. And it is not good at filtering out noise.
Implementation: The algorithm is straightforward and easy to implement	

Hough transform

Describe your implementation. How does it divert from what is explained in the book? Did you modify it so that complex cases are handled? Did you encounter any unexpected results? Eg. planes fitted to points where you don't want it. Show examples and explain why that happened.

Main implementation:

Our implementation utilizes a Randomized Hough Transform (RHT) approach for plane detection in 3D point clouds. The RHT method improves upon the computational efficiency of traditional Standard Hough Transform (SHT) by randomly sampling three points from the dataset to calculate plane parameters, reducing the computational load on the accumulator.

1. **Accumulator Setup:** We initialize an accumulator, which is a 3D matrix with parameters n_theta , n_phi , and n_rho to represent the Hough space. For example, if each of these parameters is set to 100, the accumulator will be a 100x100x100 matrix, initialized with zero values.
2. **Three points randomly sampled:**
p1 is a point randomly selected from indices.
p2 and p3 are two points randomly selected in the neighborhood of p1, ensuring that they are close.
3. **Plane parameter calculation:**
Use the three points (p1, p2, p3) to construct two vectors v1 and v2.
Use the vector cross product $np.cross(v1, v2)$ to calculate the normal vector normal.
Then, calculate the plane parameter d by normalizing the normal vector and combining it with the coordinates of p1.
4. **Accumulator update:**
Calculate the spherical coordinate parameters (theta, phi, rho) of the plane, corresponding to the 3D index of the accumulator.
The accumulator cell corresponding to the index is then incremented once (vote).
5. step 2 to 4 are repeated for an `alpha` amount of times
6. The plane with the most votes is added. All points within 'epsilon' distance from the plane, are added to the plane.
7. These steps are repeated till one of the constraints is met

Major Differences from the Book:

Standard Hough Transform in the Book calculates plane parameters for all points and increments all relevant accumulator cells. While accurate, this method is computationally expensive.

Randomized Hough Transform in Our Code reduces computational overhead by focusing on subsets of the data. And accumulator cells are incremented only for the sampled planes, making the process faster and less memory-intensive. Additionally, our code integrates clustering (e.g., DBSCAN) to filter noisy inliers, which is not discussed in the book.

Problem: Detecting the Same Plane as Two Separate Ones

Some of the planes would be separated into 2 planes. With the right use of parameters (`epsilon` not too low and enough bins in the accumulator) this can be reduced but not solved completely. The attempts to merge the 2 planes together resulted in a very long runtime, so were not implemented.

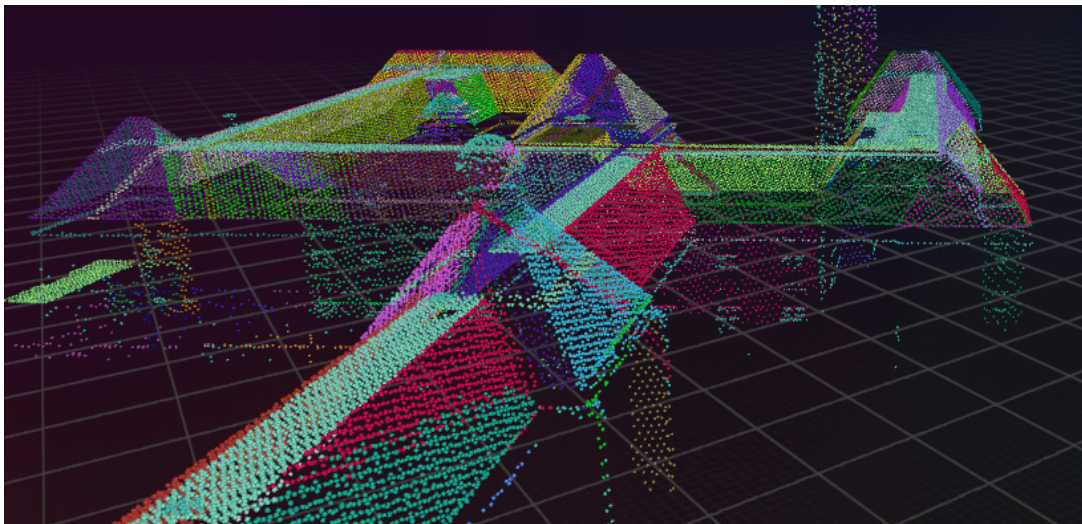


Figure 1: RHT is based on random sampling, which may cause the same plane to be incorrectly divided into multiple parts if the points on the wall are unevenly distributed due to noise or sparse point clouds.

Problem: Intersection of planes

When planes intersect it can create the situation where plane 1 is extended as a line through plane 2. We did not find a way to prevent this. The only way to limit this effect is to reduce `epsilon`.

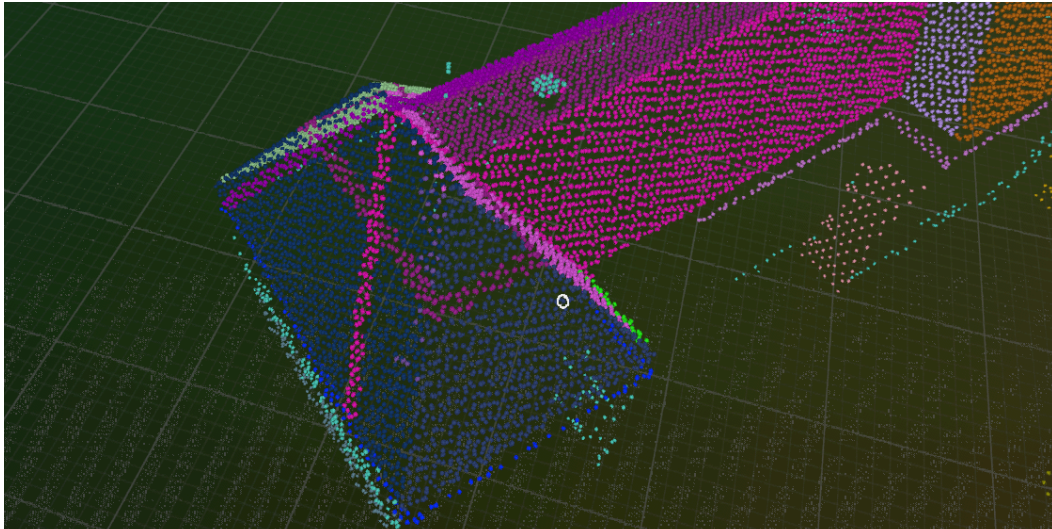


Figure 2. problems with intersecting planes

Problem: divided planes

When 2 parts of the roof are on the same plane they will be part of the same plane even if they are far apart and not connected. To prevent this, DBSCAN was used to find the different clusters and only use the points of the biggest cluster.

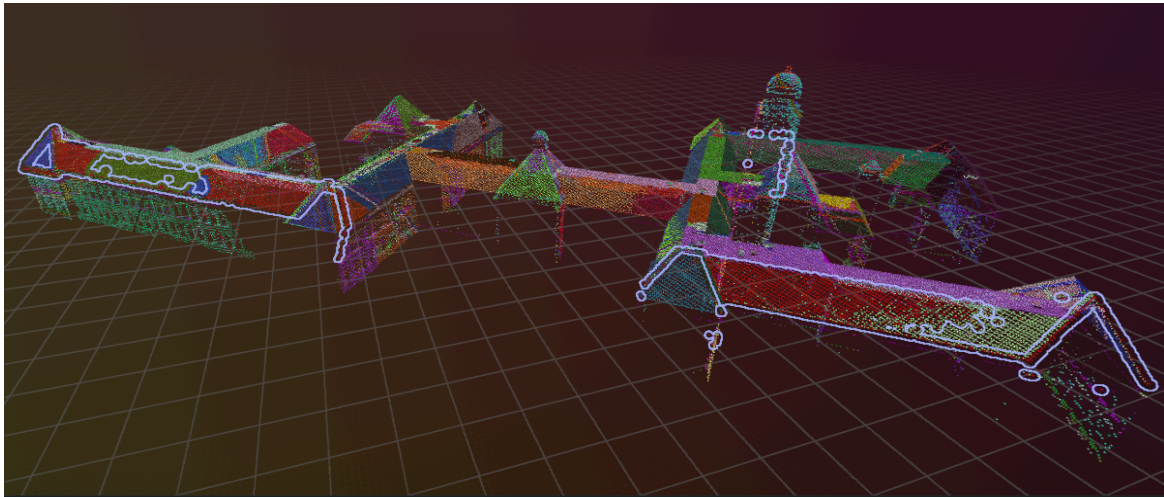


Figure 3. Problem with divided planes

What parameters are necessary and exposed to the user? (mention if you added more than the ones given in the starting code)

```
"HoughTransform": {
  "alpha":,
  "epsilon":,
  "n_theta":,
  "n_phi":,
  "n_rho":,
  "neighborhood_radius_1stplane":,
  "max_inlier_distance":,
  "n_samples":,
  "max_ins_points_in_plane_counts":
```

1. **Alpha**: Minimum number of points required to form a plane.
2. **Epsilon**: The maximum distance a point can deviate from a plane and still be considered part of it. A smaller epsilon reduces the impact of intersecting planes but may not categorize walls correctly if the distance between points increases due to a lower point density and the ornament of the walls.
3. **n_theta, n_phi, n_rho**: The number of bins in the θ (theta), ϕ (phi) and ρ (rho) direction for the accumulator.
4. **neighborhood_radius_1stplane**: The maximum distance the second and third random points can be from the first point.
5. **max_inlier_distance**: Maximum allowed distance between two separated areas to still be considered part of the same plane. This should not be smaller than epsilon to avoid cutting planes at the intersection of two planes.
6. **n_samples**: Number of sample points to choose randomly. A higher number provides better results but also increases runtime.
7. **max_ins_points_in_plane_counts**: Maximum number of consecutive times the restriction (that there aren't enough points to form a plane) can be triggered. This prevents an infinite loop when the remaining points cannot form a plane and avoids terminating the program too early after just one failed attempt.

Interactions Between Parameters:

Alpha vs. n_samples: If alpha is set high and n_samples is low, the algorithm might fail to form planes with enough points. In this case, the `max_ins_points_in_plane_count` should be increased to prevent the program from stopping too quickly after just one failed plane detection.

n_samples vs. n_theta, n_phi, n_rho: The number of bins (n_theta, n_phi, n_rho) and the number of samples are interrelated. A higher number of bins provides a finer resolution for plane detection, allowing planes to be better fitted. However, if alpha is too low compared to the number of bins, the votes will be too spread out, and planes could be incorrectly formed with only 2 or 3 votes, even though better planes exist.

Show the results and provide the best parameters for the BK-City dataset. Also, how did you choose the optimal parameters?

1. **Epsilon**: Set to **0.4**. This value ensures that planes, especially walls, are detected correctly without being split due to surface ornamentation. Lowering epsilon further would fail to categorize the walls, as the distances between points on these walls are slightly larger.
2. **Alpha**: Set to **50**. This value is high enough to ensure the detection of larger, more significant planes, while avoiding the detection of small, irrelevant details.
3. **max_inlier_distance**: Set to **1.5**. This value is at least twice the size of epsilon, ensuring that planes intersect correctly without being mistakenly separated by minor gaps.

4. `n_samples`: Set to **4000**. This provides a stable detection process without overly long runtimes.
5. `max_ins_points_in_plane_count`: Set to **100**. This prevents the algorithm from stopping prematurely if planes with fewer than `alpha` points are consistently detected, allowing the program to continue searching for valid planes.

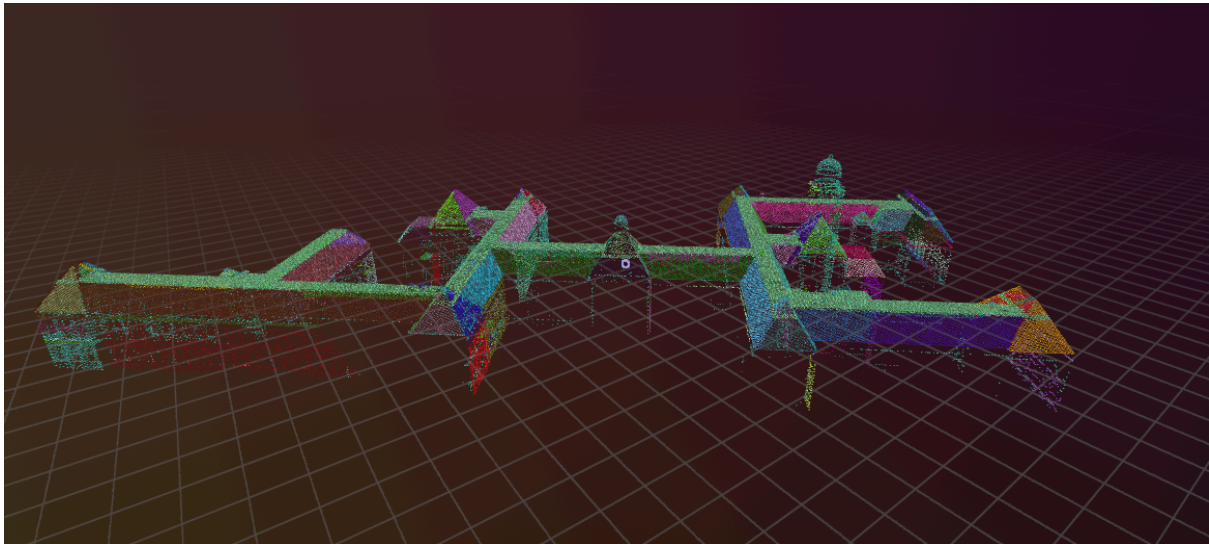


Figure 4. Final result

```
"HoughTransform": {
  "alpha": 50,
  "epsilon": 0.4,
  "n_theta": 360,
  "n_phi": 360,
  "n_rho": 360,
  "neighborhood_radius_1stplane": 2.0,
  "max_inlier_distance": 1.5,,
  "n_samples": 4000,
  "max_ins_points_in_plane_counts": 100
```

List the pros/cons of the method.

cons: lines and runtime

Pros	Cons
Efficiency: It can be quite efficient thanks to the flexible adjustment of the set of params. (when you want to speed up the process, you just make the resolution and the number of samples lower)	Complex parameter tuning: Although providing a large number of parameters enhances flexibility, it also increases the complexity of parameter adjustment. Users need to find the best parameter combination suitable for the data set through multiple experiments.
Memory Consumption: The memory usage of the accumulator is controllable, and	Sensitive to sparse point clouds or noise:

memory can be saved by adjusting parameters (such as reducing the resolution).	Despite the addition of DBSCAN and other filtering mechanisms, for sparse point clouds or severe noise, false planes may still be detected or real planes may be missed. Especially in the boundary area, improper setting of epsilon and max_inlier_distance may lead to discontinuous segmentation.
	Run time is positively correlated with accumulator size: The run time increases significantly with the increase of point cloud size or accumulator resolution. In particular, the algorithm may be limited by computational bottlenecks at larger n_theta, n_phi, and n_rho settings.
	Randomness of results: The nature of random sampling may lead to unstable results, especially when the data points are sparse, some planes may be missed. The results of multiple runs may be different, depending on the setting of the random number seed.

Which method is the best

Comparison matrix	RANSAC	Region Growing	Hough Transform
Time Consumption +++ : short elapsed time	- 5~7 min	+++ 20~40 secs	- 15 min
Memory Consumption	98 MB	120 MB	30~50 MB
How many user Defined Parameter are necessary	5	4	9
Parameter Sensitivity - High sensitivity to many parameters +++ Sensitivity to only a few	+++ Only two parameters are very sensitive	+++ The max_angle defines the normal vector deviation threshold. The minimum_region_size sets the threshold for valid	- There are 9 parameters, which is hard for users to make adjustments. And the method is sensitive to the

parameters		planar regions.	params
Intuitivity of parameters +++ users don't need much knowledge on the concept of the algorithm - users need to know the algorithm well	+++ All parameters are very easy to set. Number of iteration is something straightforward to be grasped	+ If users don't understand the concept of normal vectors or k nearest neighborhood, it will be difficult for them to set.	- epsilon and n_samples are intuitive and easy to understand. However, alpha and n_theta require Hough transform knowledge, and clustering/filtering parameters depend heavily on data, making them harder to tune intuitively.
Intersected Planes +++: no intersect planes - intersected planes	- Given its random nature, points from different planes will be added erroneously	+++ There are no intersected planes	- When the epsilon value is large, it will cause the plane to be inserted into the other plane in a line at the boundary of the other plane.
Handling of varied point density +++ point density doesn't matter - point density matter	- This makes the <code>min_score</code> parameter tricky, low point density will not be detected with low <code>min_score</code>	- Region growing relies on neighborhood points, so if there aren't enough points in the neighborhood, the segmentation accuracy will be low.	+ Point density doesn't matter a lot except if it gets so low the amount of points are lower than alpha but then the alpha can just be changed
Handling of Outliers +++ filtering out outliers easily - struggling to filter out outliers	+++ Outliers are easily left out, unless the <code>epsilon</code> is too large	- Region growing can't tell outliers from small planes	- Hough is hard to tell the outliers away from the planes (too sensitive to params)
Over Segmentation (too many planes) / Under Segmentation (too less planes) +++ less errors - many errors	+++ This is easily fixed with reasonable amount of <code>iterations</code> and reasonable <code>min_score</code>	+++ It depends on the <code>max_angle</code> and region size parameters, if correctly set, there won't be much.	- over segmentation is a problem

Edges & Corners Handling +++ edges and corners are classified correctly - difficulty in handling edge and corner points	+++ Corners are neatly defined	- It is poor at edges/corners handling	+ The epsilon determines the thickness of the plains, and so also the amount of points that will be incorrectly added to the plain after a corner
How much post-processing was necessary +++ little modification - a lot of modification	++ Not many modifications were necessary	+ Only for normal vector calculation and seed point selection	+ The most tricky and useful part is the adjustment of params
Multi-scale Dataset Handling +++ good segmentation regardless of the dataset size - good performance only on small dataset	++ Different Dataset might cause new problems because of the size parameters	+++ Good segmentation result on large and small dataset	+ Actually the same, as it mainly depends on the density and shape of the point cloud (works well for the smaller plane detection)
Sum	16	14	-2

Final considerations

RANSAC's simplicity in implementation and parametrization makes it an accessible option for users, although this comes at the cost of a longer runtime. Moreover, the results are generally quite reasonable.

Region Growing is a very compelling technique that delivers precise results for this assignment. While its implementation can be slightly challenging due to its complexity, its relatively short runtime makes it an efficient choice. However, in real life scenarios (i.e. architectural surface segmentation), there will be a lot of noise in the collected data, and region growing isn't good at filtering out the noise, thus the calculation of normal vectors will be challenging.

The Hough Transform might be a robust technique. However, its complex implementation may make it less efficient. Additionally, it requires a significant computational effort, resulting in a long runtime.

Choice

Based on the comparison matrix and final considerations, RANSAC appears to be the preferred choice due to its balance of efficiency and ease of implementation. Although we have not been able to finalise its post-processing, it seems possible to achieve high accuracy fairly easily. Its practical simplicity and reasonable runtime make it a better option for scenarios where overall efficiency is a key factor.

Workload subdivision

The workload was roughly divided as shown in the table below, however we did discuss together to solve problems encountered.

RANSAC: Carmem

Region growing: Xinya

Hough transform: Xu and Nicolette